

Beginner — Scenarios 1–8

1. Student Grade Calculator

Create an array of student objects:

```
[
  { name: "Arun", marks: 85 },
  { name: "Priya", marks: 72 },
  { name: "Kumar", marks: 45 }
]
```

Requirements:

- Calculate grades using conditions.
- Loop through all students.
- Create a function `calculateGrade(marks)`.
- Display each student's name, marks, and grade.

Explanation:

You need to process multiple student objects stored in an array. Loop through each student, pass their marks to the `calculateGrade()` function, use conditions to determine the grade, and display the student's name, marks, and calculated grade.

Practice: Objects + Arrays + Loops + Conditions + Functions

2. Employee Salary Calculator

Create employee objects:

```
{
  name: "John",
  salary: 50000,
  experience: 6
}
```

Calculate:

- Experience < 3 → 5% bonus
- Experience 3–5 → 10%
- Experience > 5 → 15%

Create a function:

```
calculateSalary(employee)
```

Explanation:

You need to calculate an employee's final salary based on their experience. The function should receive the employee object, check the experience using conditions, calculate the appropriate bonus percentage, and return or display the final salary.

Practice: Objects + Functions + Conditions + Calculations

3. Product Price Calculator

Create an array of products:

```
[
  { name: "Laptop", price: 60000 },
  { name: "Mouse", price: 1500 },
  { name: "Keyboard", price: 3000 }
]
```

Requirements:

- Calculate total price.
- Apply discount:
 - Total > ₹50,000 → 20%
 - Total > ₹20,000 → 10%
 - Otherwise → no discount.
- Display final amount.

Explanation:

Loop through all products and calculate their combined price. Once you have the total, use conditions to determine which discount applies, calculate the discount amount, and then calculate the final amount.

Practice: Objects + Arrays + Loops + Conditions + Functions

4. Find Largest Number

Given:

```
[25, 10, 75, 42, 90, 18]
```

Create a function:

findLargest(numbers)

Do **not** use `Math.max()`.

Use:

- Array
- Loop
- Condition
- Function

Explanation:

Start by assuming the first number is the largest. Loop through the remaining numbers and compare each one with the current largest value. Whenever you find a bigger number, update the largest value.

Practice: Arrays + Loops + Conditions + Functions

5. Find Even and Odd Numbers

Given:

[10, 23, 44, 51, 67, 82, 90]

Create a function that separates numbers into:

```
{
  even: [],
  odd: []
}
```

Example:

```
{
  even: [10, 44, 82, 90],
  odd: [23, 51, 67]
}
```

Explanation:

Loop through every number and check whether it is divisible by 2. If the remainder is 0, add it to the `even` array; otherwise, add it to the `odd` array.

Practice: Arrays + Loops + Conditions + Functions

6. Shopping Cart Total

Create:

```
const cart = [  
  { name: "Laptop", price: 60000, quantity: 1 },  
  { name: "Mouse", price: 1500, quantity: 2 },  
  { name: "Keyboard", price: 3000, quantity: 1 }  
];
```

Calculate:

- Item total = price × quantity
- Cart total
- Then apply discount based on total.

Explanation:

Each item has a price and quantity. Loop through the cart, calculate each item's total, and add all item totals to get the cart total. Then use conditions to apply a discount based on the total amount.

Practice: Objects + Arrays + Loops + Conditions + Functions

7. Login Validation

Create a user object:

```
{  
  username: "admin",  
  password: "admin123"  
}
```

Create:

```
login(username, password)
```

Conditions:

- Correct username + password → Login successful
- Correct username + wrong password → Invalid password

- Wrong username → User not found

Explanation:

Compare the entered username and password with the stored user object. Use conditions to determine whether both values are correct, only the username is correct, or the username does not exist.

Practice: Objects + Conditions + Functions

8. Temperature Converter

Create functions:

celsiusToFahrenheit()
fahrenheitToCelsius()

Use conditions to determine whether the temperature is:

- Very Cold
- Cold
- Normal
- Hot
- Very Hot

Explanation:

Create functions to convert temperatures between Celsius and Fahrenheit. After conversion, use conditions to classify the temperature into one of the given categories.

Practice: Functions + Conditions + Calculations

Intermediate — Scenarios 9–17

9. Employee Search System

Create an employee array:

```
[  
  { id: 101, name: "Arun", department: "IT" },  
  { id: 102, name: "Priya", department: "HR" },  
  { id: 103, name: "John", department: "IT" }  
]
```

```
]
```

Create functions:

```
findEmployeeById(id)
findEmployeesByDepartment(department)
```

Return matching employee objects.

Explanation:

Search through the employee array using loops. One function should find a single employee based on ID, while another function should collect and return all employees belonging to a specific department.

Practice: Objects + Arrays + Loops + Conditions + Functions

10. Inventory Management

Create:

```
[
  { name: "Laptop", stock: 10 },
  { name: "Mouse", stock: 0 },
  { name: "Keyboard", stock: 5 }
]
```

Create functions:

```
checkStock()
getOutOfStockProducts()
getLowStockProducts()
```

Rules:

- `stock === 0` → Out of Stock
- `stock <= 5` → Low Stock
- `stock > 5` → Available

Explanation:

Analyze the stock value of every product and classify it based on the rules. Create separate functions to identify products that are out of stock or running low.

Practice: Objects + Arrays + Loops + Conditions + Functions

11. Bank Account Transaction System

Create:

```
const account = {  
  name: "Murali",  
  balance: 50000,  
  transactions: []  
};
```

Create functions:

```
deposit()  
withdraw()  
getBalance()
```

Rules:

- Deposit must be positive.
- Withdrawal cannot exceed balance.
- Store every transaction in `transactions`.

Example transaction:

```
{  
  type: "deposit",  
  amount: 5000  
}
```

Explanation:

Build a simple banking system where the account object stores the balance and transaction history. The deposit function increases the balance, while the withdraw function validates the available balance before deducting money. Every successful transaction should be stored.

Practice: Objects + Arrays + Functions + Conditions + Validation

12. Expense Tracker

Create:

```
[  
  { category: "Food", amount: 500 },  
  { category: "Travel", amount: 1000 },  
  { category: "Food", amount: 700 },  
]
```

```
{ category: "Shopping", amount: 2000 }  
]
```

Create functions:

```
calculateTotal()  
getFoodExpenses()  
getHighestExpense()
```

Also calculate:

- Food total
- Travel total
- Shopping total
- Overall total

Explanation:

Loop through all expenses and calculate totals. Track each category separately, calculate the overall spending amount, and compare expenses to identify the highest individual expense.

Practice: Arrays + Objects + Loops + Conditions + Functions

13. Employee Attendance System

Create:

```
[  
  { name: "Arun", presentDays: 22, totalDays: 25 },  
  { name: "Priya", presentDays: 18, totalDays: 25 }  
]
```

Calculate attendance percentage.

Rules:

- = 90% → Excellent
- = 75% → Good
- = 60% → Average
- < 60% → Poor

Create:

```
calculateAttendance(employee)
```


Explanation:

Calculate each employee's attendance percentage using:

$(\text{presentDays} / \text{totalDays}) \times 100$

Then use conditions to classify the employee's attendance performance.

Practice: Objects + Functions + Conditions + Calculations

14. Product Search & Filtering

Given:

```
[
  { name: "iPhone", category: "Mobile", price: 80000 },
  { name: "Samsung", category: "Mobile", price: 60000 },
  { name: "Dell", category: "Laptop", price: 70000 },
  { name: "HP", category: "Laptop", price: 50000 }
]
```

Create functions:

```
findByCategory()
findByPriceRange()
findProductByName()
```

Example:

```
findByPriceRange(50000, 70000)
```

Explanation:

Search the product array using different criteria. One function searches by category, another checks whether prices fall within a range, and another searches by product name.

Practice: Arrays + Objects + Loops + Conditions + Functions

15. Order Status System

Create orders:

```
[
  { id: 1, customer: "Arun", status: "shipped" },
  ...
]
```

```
{ id: 2, customer: "Priya", status: "pending" },  
{ id: 3, customer: "John", status: "delivered" }  
]
```

Create a function:

```
getOrderMessage(order)
```

Return:

- pending → Order is being processed
- shipped → Order is on the way
- delivered → Order delivered successfully
- cancelled → Order cancelled

Explanation:

The function receives an order object and checks its status. Based on the status value, it returns the appropriate user-friendly message.

Practice: Objects + Conditions + Functions

16. Shopping Cart Validation

Given:

```
[  
  { name: "Laptop", price: 60000, quantity: 1 },  
  { name: "Mouse", price: 1500, quantity: 0 },  
  { name: "Keyboard", price: 3000, quantity: 2 }  
]
```

Create functions:

```
validateCart()  
calculateCartTotal()  
getInvalidItems()
```

Rules:

- Quantity must be greater than 0.
- Price must be greater than 0.
- Product name cannot be empty.

Explanation:

Before calculating the total, validate every item in the cart. Check each product's name, price, and quantity. Invalid products should be identified separately, while valid products can be used for calculations.

Practice: Validation + Objects + Arrays + Loops + Conditions + Functions

17. Student Performance Analyzer

Create:

```
[
  {
    name: "Arun",
    marks: [80, 75, 90, 85]
  },
  {
    name: "Priya",
    marks: [60, 70, 65, 72]
  }
]
```

Create functions:

```
calculateAverage()
findHighestMark()
findLowestMark()
calculateGrade()
```

Generate:

```
{
  name: "Arun",
  average: 82.5,
  highest: 90,
  lowest: 75,
  grade: "A"
}
```

Explanation:

Each student contains an array of marks. You need to process this nested array to calculate the average, highest mark, lowest mark, and grade. Finally, create a summary object containing all the calculated information.

Advanced — Scenarios 18–25

18. E-Commerce Order Processing

Create an order:

```
{
  customer: {
    name: "Murali",
    city: "Bangalore"
  },
  items: [
    { name: "Laptop", price: 60000, quantity: 1 },
    { name: "Mouse", price: 1500, quantity: 2 }
  ],
  paymentMethod: "UPI"
}
```

Create functions:

```
calculateSubtotal()
calculateDiscount()
calculateTax()
calculateFinalAmount()
validateOrder()
```

Final output:

```
{
  subtotal: 63000,
  discount: 6300,
  tax: 10206,
  finalAmount: 66906
}
```

Explanation:

Process a complete customer order step by step. First validate the order, calculate the subtotal from all items, calculate the discount, calculate tax, and finally determine the final amount payable.

Suggested Flow:

Validate Order → Calculate Subtotal → Apply Discount → Calculate Tax → Final Amount

Practice: Nested Objects + Arrays + Loops + Conditions + Multiple Functions

19. Employee Department Analyzer

Given 20+ employees:

```
[
  {
    name: "Arun",
    department: "IT",
    salary: 80000,
    experience: 6
  },
  ...
]
```

Create functions to determine:

- Highest-paid employee
- Lowest-paid employee
- Average salary
- Employees by department
- Employees with > 5 years experience
- Employees earning > ₹100,000

Avoid built-in methods initially. Use loops and conditions.

Explanation:

Analyze a larger employee dataset and generate useful insights. Use loops to compare salaries, calculate totals, identify employees based on experience, and group or collect employees based on department.

Practice: Large Arrays + Objects + Loops + Conditions + Functions + Data Analysis

20. Banking Transaction Analyzer

Given:

```
[
  { type: "credit", amount: 5000 },
  { type: "debit", amount: 2000 },
  { type: "credit", amount: 10000 },
  { type: "debit", amount: 3000 }
]
```

Create:

```
calculateBalance()
getTotalCredits()
getTotalDebits()
getLargestTransaction()
```

Output:

```
{
  totalCredits: 15000,
  totalDebits: 5000,
  balance: 10000
}
```

Explanation:

Loop through all transactions and check whether each transaction is a credit or debit. Add amounts to separate totals, calculate the final balance, and compare amounts to find the largest transaction.

Practice: Arrays + Objects + Loops + Conditions + Functions

21. Duplicate Data Detector

Given:

```
[
  { id: 101, email: "a@test.com" },
  { id: 102, email: "b@test.com" },
  { id: 103, email: "a@test.com" }
]
```

Create:

```
findDuplicateEmails(users)
```

Return:

```
["a@test.com"]
```

Then extend it to detect duplicate IDs.

Explanation:

Compare user data to identify repeated email addresses. Make sure the duplicate value appears only once in the result. Then apply the same logic to check for duplicate IDs.

Practice: Arrays + Objects + Loops + Conditions + Comparison Logic

22. Inventory Reorder System

Given:

```
[
  {
    name: "Laptop",
    stock: 3,
    reorderLevel: 5,
    reorderQuantity: 10
  },
  {
    name: "Mouse",
    stock: 20,
    reorderLevel: 5,
    reorderQuantity: 10
  }
]
```

Create:

`generateReorderList(products)`

If:

`stock <= reorderLevel`

add the product to the reorder list.

Expected:

```
[
  {
    name: "Laptop",
```

```
    orderQuantity: 10
  }
]
```

Explanation:

Check every product's current stock against its reorder level. If stock is too low, create a new object containing the product name and the quantity that needs to be ordered.

Practice: Objects + Arrays + Loops + Conditions + Functions

23. Multi-Level Employee Salary Analysis

Create:

```
[
  {
    name: "Arun",
    department: "IT",
    salary: 100000,
    performance: {
      rating: 5
    }
  },
  {
    name: "Priya",
    department: "HR",
    salary: 80000,
    performance: {
      rating: 3
    }
  }
]
```

Bonus rules:

- Rating 5 → 20%
- Rating 4 → 15%
- Rating 3 → 10%
- Rating < 3 → 5%

Create:

```
calculateBonus(employee)
calculateFinalSalary(employee)
```


Then find the employee receiving the highest bonus.

Explanation:

Access the nested `performance.rating` property for each employee. Use the rating to determine the bonus percentage, calculate the bonus amount and final salary, then compare all employees to identify who receives the highest bonus.

Practice: Nested Objects + Arrays + Loops + Conditions + Functions

24. Complete Shopping Cart Engine

Build a mini shopping-cart engine using only the five topics.

Your data:

```
const cart = [
  {
    product: {
      id: 1,
      name: "Laptop",
      price: 60000
    },
    quantity: 1
  },
  {
    product: {
      id: 2,
      name: "Mouse",
      price: 1500
    },
    quantity: 2
  }
];
```

Implement:

```
addProduct()
removeProduct()
updateQuantity()
calculateSubtotal()
calculateDiscount()
calculateTax()
calculateFinalTotal()
```

Add business rules for:

- Discount
- Tax
- Invalid quantity
- Out-of-stock products
- Minimum order amount

This is an excellent interview-style exercise.

Explanation:

Build a complete mini shopping cart system. The cart should support adding, removing, and updating products while validating business rules. After managing the products, calculate subtotal, discount, tax, and final payable amount.

Practice: Nested Objects + Arrays + Loops + Conditions + Functions + Business Logic

25. Mini Employee Management System ★

Build a complete console-based employee management system.

Employee:

```
{  
  id: 101,  
  name: "Arun",  
  department: "IT",  
  salary: 90000,  
  experience: 6  
}
```

Implement:

```
addEmployee()  
removeEmployee()  
findEmployee()  
updateEmployee()  
getEmployeesByDepartment()  
getHighestSalaryEmployee()  
getLowestSalaryEmployee()  
calculateAverageSalary()  
getExperiencedEmployees()  
calculateBonus()
```

Business rules:

- Experience $\geq 10 \rightarrow 20\%$ bonus
- Experience $\geq 5 \rightarrow 15\%$ bonus
- Experience $\geq 3 \rightarrow 10\%$ bonus
- Experience $< 3 \rightarrow 5\%$ bonus

Then generate an employee report:

Employee Report

Total Employees: 10
Average Salary: ₹85,000
Highest Salary: ₹1,50,000
Lowest Salary: ₹35,000
IT Employees: 4
HR Employees: 3
Experienced Employees: 6
Total Bonus: ₹1,25,000

Explanation:

Build a complete employee management application using all five JavaScript topics. The system should allow you to add, remove, search, and update employees. You should also analyze employee data, calculate salaries and bonuses, count employees by department, and generate a final report.

Suggested Flow:

Manage Employees $\rightarrow$ Validate Data $\rightarrow$ Analyze Employees $\rightarrow$ Calculate Bonuses $\rightarrow$ Generate Report

Practice:

- Objects
- Arrays
- Loops
- Conditions
- Functions
- CRUD-like operations
- Validation
- Calculations
- Business rules
- Report generation

